

AI-Usage Detection: Codebook and Interpretation Guide

DA matching experiment (oTree) — IRB-44866

July 30, 2026

Contents

1. What this measures	1
What it cannot see	2
2. Where the data lives	2
3. Field reference	2
3.1 Summary fields	2
3.2 Attention and presence	2
3.3 Clipboard	3
3.4 Typing dynamics on the free-text answer	3
3.5 Environment	3
4. Flag definitions	4
5. Interpretation	4
5.1 The first rule: do not threshold on <code>ai_score</code>	4
5.2 Flags tiered by precision	4
5.3 Look for the sequence, not the flag	4
5.4 Reading the keystroke ratio	5
6. Confounds and false positives	5
6.1 The base-rate problem	5
7. Recommended analysis workflow	6
7.1 Calibrate before trusting any threshold	6
8. Ethics and policy	6
9. Implementation reference	6

1. What this measures

Participants recruited online may use ChatGPT or a similar assistant to play the experiment. Because the study is about how *people* reason about matching mechanisms, an AI-generated ranking is noise presented as data.

The instrumentation records passive browser behaviour on the instructions, decision, and survey pages, and converts it into a set of **soft research flags** for post-hoc exclusion.

Nothing in this system blocks a participant, alters their page flow, or changes their payment. It is deliberately kept separate from the pre-existing honeypot path (suspected_bot -> BotBlocked -> Prolific redirect), which remains the only hard block in the study.

These signals are not grounds for rejecting a Prolific submission. They are analysis-time exclusion flags only.

What it cannot see

No browser instrumentation can detect any of the following. Treat every number in this codebook as a **lower bound** on AI use.

- A phone beside the laptop. A participant reading the instructions off the screen and typing into a chatbot on their phone shows a perfectly normal keystroke ratio and no clipboard activity. This is the residual threat.
- Native OS screenshots (Cmd+Shift+4, Win+Shift+S) — the keystrokes never reach the page.
- A second monitor, or the contents of any other tab or window.
- Clipboard contents absent a paste into our page.

2. Where the data lives

Location	Contents
Admin -> Data -> “All apps” wide CSV	participant.ai_* — accumulated totals plus ai_pages, the per-page ledger
survey app CSV, admin Data tab	survey.1.player.ai_* — the same summary, mirrored for convenience
Each treatment app’s CSV	ai_tel_* — the raw JSON blob per page, for re-deriving anything
Admin -> Data -> Page times	oTree’s built-in per-page dwell time (not duplicated here)

Instrumented pages: InstructionsQuiz and all five Decision rounds in each of the four treatment apps, plus survey/Demographics — seven pages per participant. Consent, the router, and the terminal pages are deliberately not instrumented.

3. Field reference

3.1 Summary fields

Field	Meaning
ai_score	Count of flags fired (0–9). A triage aid, not a severity measure
ai_flags	Comma-separated flag names
ai_pages_instrumented	Pages that reported telemetry; normally 7
ai_bad_blobs	Pages whose payload was missing or unparseable
ai_pages	Per-page ledger (JSON). Where the sequencing evidence lives

3.2 Attention and presence

Field	Mirror name	Meaning
ai_blur	ai_blur_count	Window blur events (focus left the browser window)
ai_hid	—	Times the tab became hidden
ai_hid_ms	ai_hidden_seconds	Total time the tab was hidden
ai_hid_max	ai_hidden_max_seconds	Longest single absence
ai_mouse, ai_scroll, ai_touch, ai_click	ai_pointer_events	Throttled activity counts
ai_loads	—	Page loads, including reloads and validation re-renders

3.3 Clipboard

Field	Mirror name	Meaning
ai_copy, ai_cut	ai_copy_count	Copy / cut events
ai_copy_ch	ai_copy_chars	Characters copied out
ai_copy_sample	—	First 160 characters copied. Inspect this directly
ai_paste	ai_paste_count	Paste events
ai_paste_ch	ai_paste_chars	Characters pasted in
ai_sel_max	ai_max_selection_chars	Largest text selection observed
ai_jump_max	ai_max_jump_chars	Largest single-event insertion with no preceding paste event

3.4 Typing dynamics on the free-text answer

Field	Mirror name	Meaning
ai_advice_len	ai_advice_chars	Final character count
ai_advice_keys	ai_advice_keys	Keydowns in that field
ai_advice_ratio	ai_advice_key_ratio	Keydowns / characters. The most diagnostic single number
ai_advice_ttfk	ai_advice_ttfk_ms	Time to first keystroke
ai_advice_ms	ai_advice_compose_ms	Span from first to last keystroke
ai_advice_ime	ai_advice_ime_keys	IME / mobile swipe-typing keys.
—	ai_advice_words	Confound control — see §6 Word count of the submitted answer

3.5 Environment

Field	Meaning
ai_webdriver	navigator.webdriver set, or a headless user-agent
ai_env	Raw fingerprint: languages, plugins, cores, screen, viewport, timezone
ai_no_pointer_pages	Pages with zero pointer activity of any kind

4. Flag definitions

Thresholds are the `T_*` constants in `ai_detect.py`. They are **first guesses, not empirical** — see §7.

Flag	Condition
<code>tab_switch</code>	<code>ai_hid_ms >= 30000</code> or <code>ai_blur >= 3</code>
<code>long_absence</code>	<code>ai_hid_max >= 60000</code>
<code>copied_out</code>	<code>ai_copy_ch >= 200</code> or <code>ai_cut > 0</code>
<code>large_selection</code>	<code>ai_sel_max >= 1500</code>
<code>pasted_in</code>	<code>ai_paste_ch >= 100</code>
<code>paste_like_insert</code>	<code>ai_jump_max >= 40</code>
<code>low_keystroke_ratio</code>	<code>ai_advice_len >= 100</code> and <code>ratio < 0.5</code>
<code>no_pointer_activity</code>	3+ pages instrumented and zero mouse/touch/scroll
<code>automation_fingerprint</code>	<code>ai_webdriver</code> true

5. Interpretation

5.1 The first rule: do not threshold on `ai_score`

`ai_score` is an unweighted count of flags that differ enormously in diagnosticity. A participant who alt-tabbed three times scores 1; so does one running a scripted browser. **Always work from the underlying columns.**

5.2 Flags tiered by precision

Tier 1 — high precision. These justify action.

- `copied_out` — text was copied out of the page. On an instructions page there is almost no legitimate reason to do this. Check `ai_copy_sample`: if it contains your instruction text, that is about as direct as browser evidence gets.
- `large_selection` — a select-all of the instructions. Usually co-occurs with `copied_out`. A large `ai_sel_max` with `ai_copy` of 0 suggests copying through an extension or share sheet that does not fire the event.
- `automation_fingerprint` — a scripted browser, not a human. Rare but decisive.
- `no_pointer_activity` — zero mouse, scroll, and touch across the entire study. A human cannot produce this.

Tier 2 — moderate. Requires corroboration.

- `pasted_in`, `paste_like_insert`, `low_keystroke_ratio` — these three almost always fire together on a pasted free-text answer. They fire *identically* for a conscientious participant who drafted in Notes and pasted. The discriminator is whether the same participant also copied text **out** earlier (see §5.3).
- `long_absence` — a single absence of a minute or more.

Tier 3 — covariate only. Never an exclusion criterion on its own.

- `tab_switch` — by far the noisiest signal. Excluding on this alone would drop 30–50% of a normal online sample. Everyone checks email, the recruitment tab, or a message mid-study.

5.3 Look for the sequence, not the flag

Individual flags are weak. The AI workflow leaves a distinctive signature, visible in the `ai_pages` ledger:

copy on */InstructionsQuiz -> long hidden interval on that same page -> paste on survey/Demographics

Text leaves the page, time passes off-page, text arrives back. This closed loop is worth considerably more than any single flag. A participant who merely pasted their own draft will not show the copy-out leg.

5.4 Reading the keystroke ratio

`ai_advice_key_ratio` = keydowns / final characters.

Ratio	Interpretation
0.9 – 1.4	Genuine typing. Above 1.0 is normal: shift, backspace, and arrow keys all register as keydowns
0.3 – 0.7	Paste followed by substantial editing
below 0.1	Effectively a pure paste

Reference values from instrumented test runs: a simulated typist produced **1.13** (340 keydowns, 300 characters); a simulated paste produced **0.014** (6 keydowns, 420 characters).

The flag threshold of 0.5 is deliberately generous. Genuine typists rarely fall below 0.7.

6. Confounds and false positives

Every item below is real and observed in practice. They are the reason these flags must remain soft.

- **Legitimate tab switching.** The dominant confound. Treat as a covariate.
- **Drafting elsewhere, then pasting.** Trips `pasted_in`, `paste_like_insert`, and `low_keystroke_ratio` simultaneously — an innocent participant can look maximally guilty on the free-text measures. Corroborate with copy-out.
- **Mobile swipe typing and autocorrect.** Inserts many characters per input event and reports `keyCode 229`. **Always exclude participants with high `ai_advice_ime_keys` from the keystroke-ratio test**, or the flag will principally identify Android users.
- **Browser autofill and password managers.** Produce large insertions on the demographic fields, unrelated to AI.
- **iOS Safari.** Fires blur and visibility events for the share sheet, the keyboard, and app backgrounding — inflated blur counts on mobile.
- **Privacy browsers** (Brave, Firefox with `resistFingerprinting`). Report one language and no plugins. This is *not* automation; only `navigator.webdriver` and a headless user-agent feed `automation_fingerprint`, and even `webdriver` is occasionally set by managed corporate browsers.

6.1 The base-rate problem

Before excluding anyone, work through the arithmetic. Suppose 5% of the sample uses AI, and a flag is 90% sensitive and 90% specific:

- True positives: $0.05 \times 0.90 = \mathbf{4.5\%}$ of the sample
- False positives: $0.95 \times 0.10 = \mathbf{9.5\%}$ of the sample

Positive predictive value is $4.5 / (4.5 + 9.5)$, or about **32%**. **Roughly two-thirds of flagged participants are innocent.** At realistic prevalence, only Tier 1 flags survive this calculation.

7. Recommended analysis workflow

1. **Report primary results on the full sample.** Do not build exclusions into the main specification.
2. **Add a robustness column** excluding only high-confidence cases: `copied_out` *and* `pasted_in` together, or `automation_fingerprint`, or `no_pointer_activity`.
3. **Report how far the estimates move.** If they barely move, you have shown that contamination is not driving the findings — a stronger claim than presenting a sample that merely looks clean.
4. Consider `ai_hidden_seconds` as a continuous covariate rather than a cutoff.
5. Never exclude on `tab_switch` alone.

7.1 Calibrate before trusting any threshold

The `T_*` constants were set a priori. After the first pilot batch, plot the distributions of `ai_hid_ms`, `ai_copy_ch`, and `ai_advice_key_ratio`. Genuine AI users typically appear as a visibly **separate mode** rather than a tail. Place cutoffs at the gap, then record the values you used in this codebook so the exclusion rule is reproducible.

8. Ethics and policy

- Participants see a visible request not to use AI on the instructions page and above the free-text survey question. There is no covert trap, no hidden prompt, and no deception in the detection system.
- The consent form was not modified for this instrumentation. If interaction logging is not covered by the approved protocol, consider an amendment.
- `ai_copy_sample` stores up to 160 characters of copied text. In practice this is your own page text, but it should be noted in the data-management plan.
- These signals must not be used to reject Prolific submissions, and must not be wired into `suspected_bot`, `blocked_for_bot`, or the bot-redirect path.

9. Implementation reference

File	Role
<code>ai_detect.py</code>	Server side: parsing, accumulation, flag scoring, word-count rule, export mirror
<code>_static/global/ai_detect.js</code>	Client instrumentation; writes one hidden field per page
<code>_templates/global/Page.html</code>	Loads the script on every instrumented page
<code>settings.py</code>	<code>PARTICIPANT_FIELDS</code> entries that make the data exportable

A related control: the free-text advice question enforces a **50-word minimum**, validated server-side in `Demographics.error_message`. The live word counter shown to participants is a hint only and never blocks; the server is authoritative.